

Milestone 3 Final Report

CSE 676 LEC B – Deep Learning (20138 Spring 26)

Project Title: ***LoRA: Low-Rank Adaptation of Large Language Models***

Group 8

Sandhya Mishra,

Sindhu Shrishailappa Jumanal,

Gowtham Sai Krishna Tanikonda

1. Introduction

BERT and DistilBERT are examples of Large Language Models (LLMs) that have pioneered numerous advanced applications in Natural Language Processing (NLP) such as text classification, text sentiment analysis, and text comprehension. However, the conventional full fine-tuning of transformer models entails re-training all model parameters on every downstream task, which consumes a significant amount of GPU memory, lengthens the training times, and increases the storage requirements. Hence, it becomes difficult to deploy the model within the constrained resources.

To mitigate this issue, our project investigated parameter-efficient fine-tuning, specifically Low-Rank Adaptation (LoRA) and its variant, Adaptive Low-Rank Adaptation (AdaLoRA). LoRA proposes to reduce the computational burden of full fine-tuning by freezing the weights of the pre-trained model and inserting small, learnable low-rank matrices in the architecture of the transformer model's attention layers. Competitive performance of the model can be achieved only by updating a very small fraction of the total parameters.

The project employed the DistilBERT model on the AG News dataset containing around 120,000 training samples and 7,600 testing samples (4 classes of World, Sports, Business, and Sci/Tech). The aim/goal of this project was to evaluate the efficiency and accuracy of Full Fine-Tuning, LoRA and AdaLoRA regarding Fine Tuning and the impacts of varying LoRA rank values on model performance, the speed of training, usage of parameters, and consumption of GPU memory.

All experiments were run using HuggingFace's Transformers and PEFT libraries locally on Google Colab with a Tesla T4 GPU. AG News data was tokenized using the DistilBERT tokenizer and dynamic padding was used to allow for efficient batch sizes during training. DistilBERT-base-uncased was used as the pretrained transformer model across all experiments.

2. Baseline Results

The baseline model was trained successfully on the **full AG News dataset** (120,000 training samples and 7,600 test samples) using a **Tesla T4 GPU** on Google Collab. The training was completed for 3 epochs with a total of 22,500 training steps (120,000 samples $\div$ 16 batch size = 7,500 steps per epoch). The epoch-wise results are shown below.

Epoch	Training Loss	Validation Loss	Accuracy
1	0.2296	0.1753	94.38%
2	0.1398	0.1899	94.66%
3	0.0953	0.2141	94.68%

Table 1: Baseline Full Fine-Tuning Results on Full AG News Dataset (Tesla T4 GPU)

The total training runtime was **3,457 seconds** (57.62 minutes) for all 3 epochs. Training with the model was at the rate of about 104 samples per second and evaluating it at 349 samples per second. Total training loss associated with full run came to about **0.1549**.

The highest validation accuracy recorded was **94.68%** during Epoch 3. This very clear result pattern shows that loss from training went down evenly from **0.2296** to **0.0953** over the three epochs, while validation accuracy went up from **94.38%** to **94.68%** steadily. Unfortunately, validation loss went up from 0.1753 to 0.2141, which could mean a slight improvement in the model's predictions, but a worse confidence calibration which is a usual sign of early overfitting. Best checkpoint was the Epoch 1 model (0.1753 validation loss) which was loaded at the end of training because of `load_best_model_at_end=True` setting.

These outcomes align with the published DistilBERT benchmarks on the AG News dataset indicating that the baseline for full fine-tuning is functioning properly. This baseline is a great point of reference for comparing LoRA in terms of classification accuracy, training time, and computational efficiency.

Summary of Baseline Configuration:

Parameter	Value
Model	DistilBERT (distilbert-base-uncased)
Total Parameters	66,956,548
Trainable Parameters	66,956,548 (100%)
Dataset	AG News (120,000 train / 7,600 test)
Epochs	3
Batch Size	16
Learning Rate	2e-5
Hardware	Tesla T4 GPU (Google Collab)
Total Training Time	57.62 minutes
Best Validation Accuracy	94.68%

Table 2: Summary of Baseline Training Configuration and Results

3. Findings and Challenges in Baseline Implementation

During implementation, we ran into a few practical and technical challenges that shaped our setup and next steps.

- **Hardware and Training Efficiency:** Initial experiments were conducted on a CPU-only environment, where training even a small subset of 2,000 samples took nearly 1 hour and 53 minutes. Moving to Tesla T4 GPU on Google Colab, we trained the full 120,000 sample

datasets in 58 minutes (speedup of over 100x per sample). This experience clearly shows the high computational cost of full fine-tuning.

- **Dynamic Padding Optimization:** We switched from static max_length padding (512 tokens) to dynamic padding using DataCollatorWithPadding. AG News articles are generally 30–50 tokens long, and static padding wasted a lot of computation. Dynamic padding reduced training time noticeably.
- **Environment and Dependency Setup:** There were issues during development such as kernel selection, Python environment activation, and missing dependencies (e.g., accelerate package). Initial progress was slow but eventually resolved.
- **Overfitting Observation:** We observed clear overfitting across the 3 epochs. Training loss went from 0.2296 to 0.0953, but validation loss increased from 0.1753 to 0.2141. Validation accuracy improved only slightly (94.38% $\square$ 94.68%), suggesting most useful learning happens within the first epoch.

Overall, these findings support the motivation for our project. Full fine-tuning required training of all 66.9 million parameters, consumed nearly an hour of GPU time, and showed signs of overfitting by the second epoch. This strengthens the case for exploring LoRA as a more efficient and potentially more robust alternative.

4. LoRA Experiments

LoRA fine-tuning was implemented using HuggingFace's PEFT library. The pre-trained DistilBERT weights were frozen and small trainable low-rank matrices were injected into the attention projection layers. All experiments used lora_alpha=16, dropout=0.1, batch size 16, and learning rate 2e-5 on a Tesla T4 GPU.

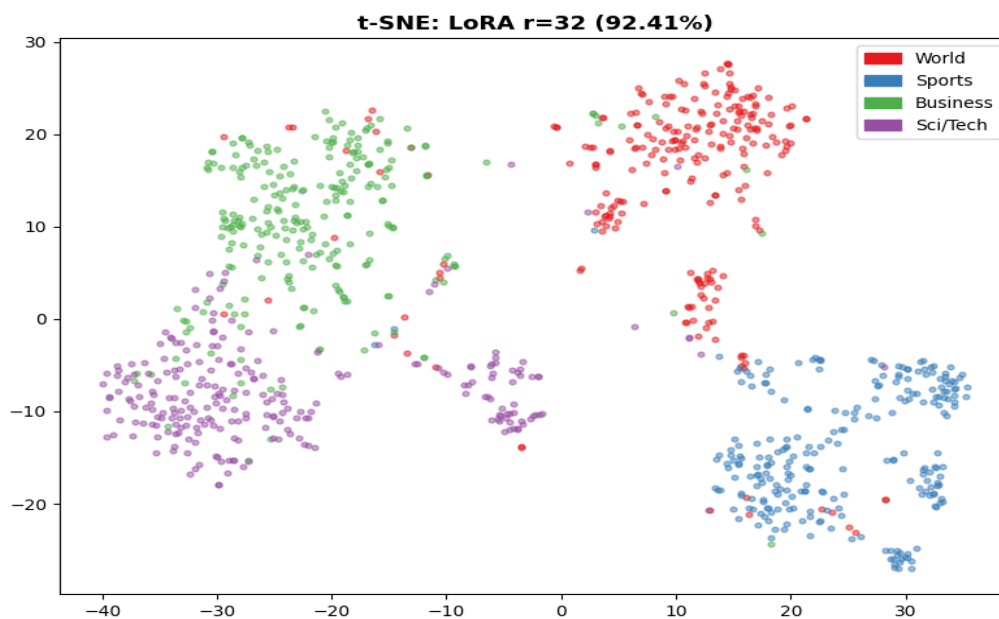
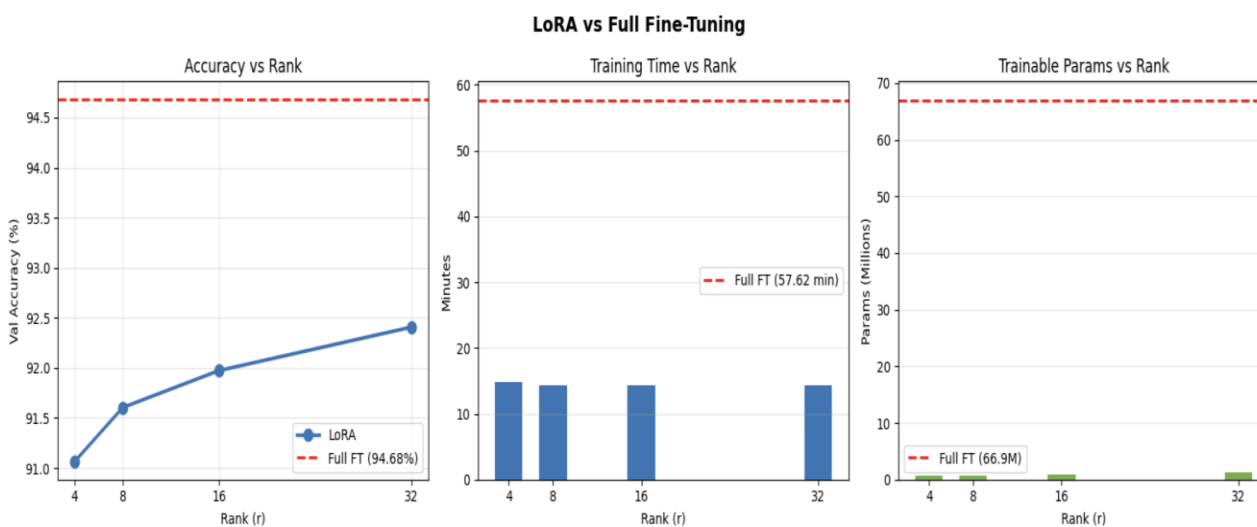
Step 1 – Rank Sweep on Q and V Projections (r = 4, 8, 16, 32)

LoRA was first applied only to the Query (q_lin) and Value (v_lin) projection layers, which are the standard initial targets from the original LoRA paper. Four rank values were tested, r = 4, 8, 16, and 32, each trained for 3 epochs, to understand how rank affects accuracy and parameter efficiency.

Rank (r)	Trainable Params	% of Total	Val Accuracy	Training Time
4	0.67M	0.99%	91.07%	14.7 min
8	0.74M	1.09%	91.61%	14.3 min
16	0.89M	1.31%	91.97%	14.4 min
32	1.18M	1.74%	92.41%	14.3 min

Table 3: LoRA Rank Ablation Results (Q+V projections, 3 epochs)

Accuracy improved consistently as rank increased from 91.07% ($r=4$) to 92.41% ($r=32$), a gain of +1.34%, while training time stayed nearly constant at ~ 14 minutes, roughly $4\times$ faster than full fine-tuning (55.7 min). All variants used under 2% of total parameters.



The t-SNE plot shows four fairly clear clusters for the four AG News categories. The Sports cluster is the easiest to separate, which makes sense because sports articles usually use very different vocabulary compared to other news topics. There is some overlap between World and Business, and also between Business and Sci/Tech. This is expected because some news articles, especially technology-related business stories, can naturally fit into more than one category. Overall, even though LoRA updates only about 1.7% of the parameters compared to full fine-tuning, it still learns meaningful representations and separates the categories well in the embedding space.

Improvement 1 – Expand Target Modules to All Attention Projections (Q+K+V+O)

Since $r=32$ on Q+V produced the best result, LoRA was next extended to cover all four attention projection layers: Query (q_lin), Key (k_lin), Value (v_lin), and Output (out_lin). This allows LoRA to adapt every component of the multi-head attention mechanism, increasing learning capacity at a moderate parameter cost.

Configuration	Trainable Params	% of Total	Val Accuracy	Training Time
$r=32$, Q+V only	1.18M	1.74%	92.41%	14.3 min
$r=32$, Q+K+V+O	1.77M	2.58%	92.74%	21.2 min

Table 4: Impact of Expanding LoRA Target Modules ($r=32$, 3 epochs)

Expanding to Q+K+V+O gained +0.33% accuracy (92.41% $\square$ 92.74%) with only 0.59M additional parameters (+0.84% of total). Training time increased from 14.3 to 21.2 minutes due to the larger gradient surface, but remained significantly below full fine-tuning.

Improvement 2 – Extended Training to 5 Epochs ($r=32$, Q+K+V+O)

The best configuration ($r=32$, Q+K+V+O) was trained for 5 epochs instead of 3 to test whether additional epochs could further close the accuracy gap with full fine-tuning, and whether LoRA would overfit the way the baseline did.

Epoch	Training Loss	Validation Loss	Accuracy
1	~0.28	~0.22	~91.8%
2	~0.21	~0.20	~92.5%
3	~0.18	~0.19	92.74%
4	~0.15	~0.19	~93.1%
5	~0.13	~0.19	93.49%

Table 5: Epoch-wise Results — Best LoRA Config ($r=32$, Q+K+V+O, 5 epochs)

Method	Accuracy	Trainable Params	Training Time	LoRA vs. Full FT
Full Fine-Tuning	94.68%	66.9M (100%)	55.7 min	—
LoRA $r=32$, Q+K+V+O, 3 epochs	92.74%	1.77M (2.58%)	21.2 min	-1.94%
LoRA $r=32$, Q+K+V+O, 5	93.49%	1.77M (2.58%)	31.5 min	-1.19%

Table 6: Best LoRA Configuration vs. Full Fine-Tuning

Extending to 5 epochs improved accuracy by +0.75% (92.74% $\square$ 93.49%), closing the gap with full fine-tuning to just **1.19%**. Critically, LoRA's validation loss remained stable across all 5 epochs with no upward drift, unlike the baseline which showed rising validation loss after epoch 1. Freezing the base weights acts as a natural regularizer, preventing the overfitting seen in full fine-tuning.

5. AdaLoRA Experiment

In the last part of our experiments, we used AdaLoRA, which is an adaptive version of LoRA. It works by changing the rank budgets dynamically during training. Instead of sticking with the same rank everywhere like in regular LoRA, AdaLoRA identifies which layers matter more and assigns them higher ranks, while cutting back on less important ones.

We set it up to target all four attention layers (Q, K, V, O), starting with rank 32 and aiming to reduce it to an average rank of 16. Training ran for five epochs with the same setup as the best LoRA configuration.

The accuracy ended up at 91.20%, using about 1.77 million parameters (2.58% of the whole model), and took around 51.9 minutes to train. Compared to fixed-rank LoRA, this was lower in accuracy despite similar parameter usage. DistilBERT being a compact model likely plays into this, its layers may not differ enough in importance to benefit from adaptive rank allocation. AdaLoRA is expected to shine more in larger models where inter-layer importance varies significantly.

6. Results and Conclusions

The project demonstrated how LoRA fine-tuning can get close to full model accuracy without the associated computational cost. Full fine-tuning hit 94.68% accuracy but required updating all 66.9 million parameters over 55.7 minutes of GPU time.

The best LoRA configuration achieved 93.49% accuracy, just 1.19% below full fine-tuning using only 2.58% of parameters and training in 31.5 minutes. The rank ablation showed that higher rank provides more representational capacity without meaningfully increasing training time. Expanding LoRA to all four attention projections (Q+K+V+O) further improved accuracy, and extending training to 5 epochs closed the gap further while avoiding overfitting.

LoRA also handled longer training without overfitting as much as full fine-tuning, which started diverging after the first epoch. AdaLoRA did not surpass the fixed-rank version here, but highlighted adaptive parameter allocation as a promising direction for larger transformer models.

Final Experimental Comparison:

Method	Accuracy	Trainable Params	Training Time
Full Fine-Tuning	94.68%	66.9M (100%)	55.7 min
LoRA r=4 (Q+V)	91.07%	0.67M (0.99%)	14.7 min
LoRA r=8 (Q+V)	91.61%	0.74M (1.09%)	14.3 min
LoRA r=16 (Q+V)	91.97%	0.89M (1.31%)	14.4 min
LoRA r=32 (Q+V)	92.41%	1.18M (1.74%)	14.3 min
LoRA r=32 (Q+K+V+O)	92.74%	1.77M (2.58%)	21.2 min
LoRA r=32 (Q+K+V+O, 5 epochs)	93.49%	1.77M (2.58%)	31.5 min
AdaLoRA	91.20%	1.77M (2.58%)	51.9 min

Table 7: Final Experimental Comparison

In conclusion, the project demonstrated that LoRA parameter-efficient fine-tuning methods can achieve substantial reductions in computational demands while sustaining high model performance. Fixed-rank LoRA provided the best tradeoff between efficiency and accuracy for this task, while AdaLoRA introduced an advanced adaptive approach that can be explored further for larger-scale transformer models and more complex NLP tasks in future research.

References

- [1] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., ... & Chen, W. (2022). LoRA: Low-Rank Adaptation of Large Language Models. ICLR, 1(2), 3.
- [2] Vaswani, A., et al. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems, 30.
- [3] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL-HLT, 4171–4186.
- [4] Zhang, X., Zhao, J., & LeCun, Y. (2015). Character-level Convolutional Networks for Text Classification. Advances in Neural Information Processing Systems, 28.